

Sri Sivasubramaniya Nadar College of Engineering, Chennai (an
autonomous institution affiliated with Anna University)

Degree & Branch	B.E. Computer Science & Engineering	Semester	V
Subject Code & Name	ICS1512 & Machine Learning Algorithms Laboratory		
Academic Year	2025-2026 (Odd)	Batch: 2023-2028	Due date:

Experiment 6: Dimensionality Reduction and Model Evaluation (With and Without PCA)

Objective

To study the effect of dimensionality reduction using Principal Component Analysis (PCA) on the performance of various machine learning classifiers. The experiment compares training and validation under two settings:

1. Without PCA (original feature space).
2. With PCA (reduced feature space).

For both cases, hyperparameter tuning and 5-fold cross-validation are applied and results are compared.

Dataset Details

- **Source:** Kaggle
- **Number of Samples:** 4602
- **Number of Features:** 58
- **Target Classes:** Binary classification (Spam vs Ham)
- **Preprocessing:** Standardization, encoding, handling missing values.

Procedure

1. Pre-process and standardize features.
2. Apply PCA (95% variance retained).
3. Define hyperparameter search grids for each model.

4. Train and validate models with 5-fold cross-validation (with and without PCA).
5. Record results and compare metrics.

Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.pipeline import Pipeline
from sklearn.ensemble import AdaBoostClassifier, GradientBoostingClassifier, RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from xgboost import XGBClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# Load dataset
data = pd.read_csv(r"C:/Users/madhu/OneDrive/Desktop/ML-3/spambase_csv.csv")

# Features and target
X = data.drop(columns=['class'])
y = data['class']

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Preprocessing and PCA
scaler = StandardScaler()
pca = PCA(n_components=0.95, random_state=42)

# Define models with optimized hyperparameter grids
models = {
    'SVM': (SVC(), {'model__C': [1], 'model__kernel': ['linear']}),
    'NaiveBayes': (GaussianNB(), {}),
    'KNN': (KNeighborsClassifier(), {'model__n_neighbors': [3]}),
    'LogisticRegression': (LogisticRegression(max_iter=1000), {'model__C': [1]}),
```

```

'DecisionTree': (DecisionTreeClassifier(random_state=42), {'model__max_depth': [10]}),
'RandomForest': (RandomForestClassifier(random_state=42), {'model__n_estimators': [10]}),
'AdaBoost': (AdaBoostClassifier(random_state=42), {'model__n_estimators': [50]}),
'GradientBoosting': (GradientBoostingClassifier(random_state=42), {'model__n_estimators': [50]}),
'XGBoost': (XGBClassifier(use_label_encoder=False, eval_metric='logloss', random_state=42), {'model__n_estimators': [50]}),
'Stacking': (StackingClassifier(estimators=[
('rf', RandomForestClassifier(random_state=42)),
('svc', SVC(probability=True, random_state=42)),
('knn', KNeighborsClassifier())
], final_estimator=LogisticRegression(max_iter=1000, n_jobs=-1), {}))
}

```

```
results = []
```

```

for model_name, (model, param_grid) in models.items():
    print(f"\nTraining {model_name} without PCA...")

```

```
# Pipeline without PCA
```

```

pipe = Pipeline([
('scaler', scaler),
('model', model)
])

```

```

grid = GridSearchCV(pipe, param_grid, cv=5, scoring='accuracy', n_jobs=-1)
grid.fit(X_train, y_train)

```

```
y_pred = grid.best_estimator_.predict(X_test)
```

```

results.append({
'Model': model_name,
'Setting': 'No PCA',
'Accuracy': accuracy_score(y_test, y_pred),
'Precision': precision_score(y_test, y_pred),
'Recall': recall_score(y_test, y_pred),
'F1 Score': f1_score(y_test, y_pred)
})

```

```
print(f"Training {model_name} with PCA...")
```

```

pipe_pca = Pipeline([
('scaler', scaler),
('pca', pca),
('model', model)
])

```

```

grid_pca = GridSearchCV(pipe_pca, param_grid, cv=5, scoring='accuracy', n_jobs=-1)
grid_pca.fit(X_train, y_train)

y_pred_pca = grid_pca.best_estimator_.predict(X_test)

results.append({
    'Model': model_name,
    'Setting': 'With PCA',
    'Accuracy': accuracy_score(y_test, y_pred_pca),
    'Precision': precision_score(y_test, y_pred_pca),
    'Recall': recall_score(y_test, y_pred_pca),
    'F1 Score': f1_score(y_test, y_pred_pca)
})

# Convert results to DataFrame
results_df = pd.DataFrame(results)

# Pivot table for comparison
pivot_df = results_df.pivot(index='Model', columns='Setting')[['Accuracy', 'Precision']]

# Split into two tables
first_half = pivot_df.iloc[:5]
second_half = pivot_df.iloc[5:]

print("\n=== Model Evaluation Comparison (First 5 Models) ===")
print(first_half)

print("\n=== Model Evaluation Comparison (Next 5 Models) ===")
print(second_half)

# Plot Accuracy Comparison
plt.figure(figsize=(12, 6))
first_half['Accuracy'].plot(kind='bar', edgecolor='black')
plt.title('Accuracy Comparison: First 5 Models (No PCA vs With PCA)')
plt.ylabel('Accuracy')
plt.xticks(rotation=45)
plt.grid(axis='y')
plt.legend(title='Setting')
plt.show()

plt.figure(figsize=(12, 6))
second_half['Accuracy'].plot(kind='bar', edgecolor='black')
plt.title('Accuracy Comparison: Next 5 Models (No PCA vs With PCA)')
plt.ylabel('Accuracy')
plt.xticks(rotation=45)

```

```

plt.grid(axis='y')
plt.legend(title='Setting')
plt.show()

# PCA Scree Plot
pca_full = Pipeline([
    ('scaler', scaler),
    ('pca', PCA(n_components=0.95, random_state=42))
])
X_transformed = pca_full.fit_transform(X_train)
explained_variance = pca_full.named_steps['pca'].explained_variance_ratio_

plt.figure(figsize=(8, 6))
plt.plot(np.cumsum(explained_variance), marker='o')
plt.xlabel('Number of Components')
plt.ylabel('Cumulative Explained Variance')
plt.title('Scree Plot: PCA Explained Variance')
plt.grid(True)
plt.show()

```

Plots

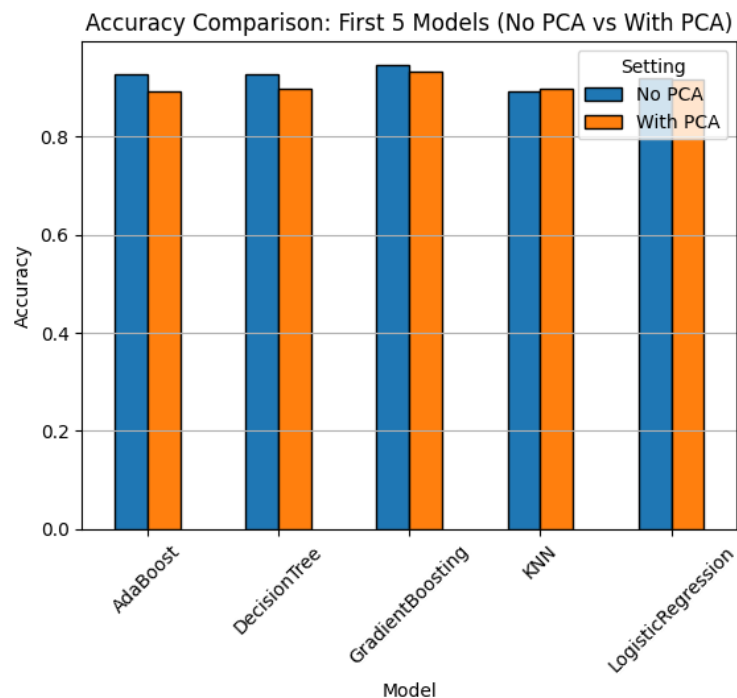


Figure 1: Model performance comparison (No-PCA vs With-PCA)

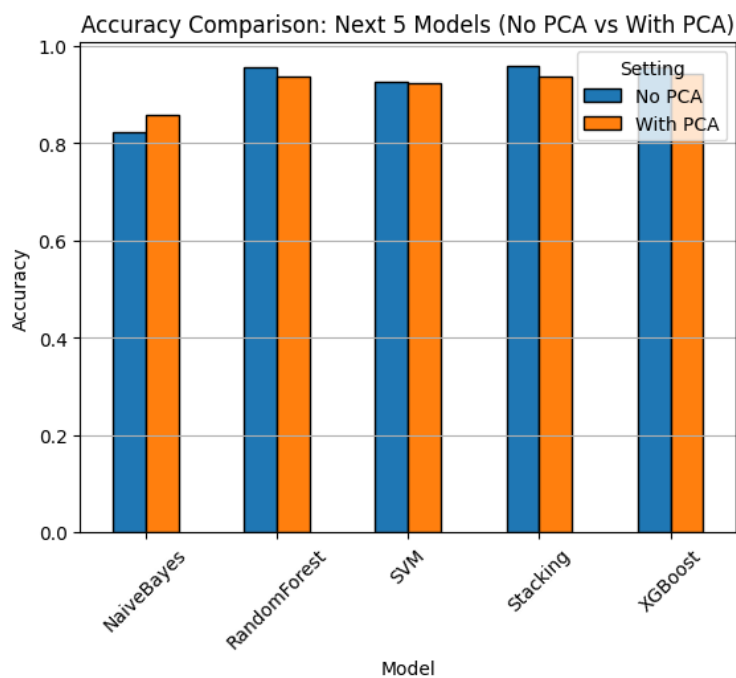


Figure 2: Cross-validation comparison across classifiers

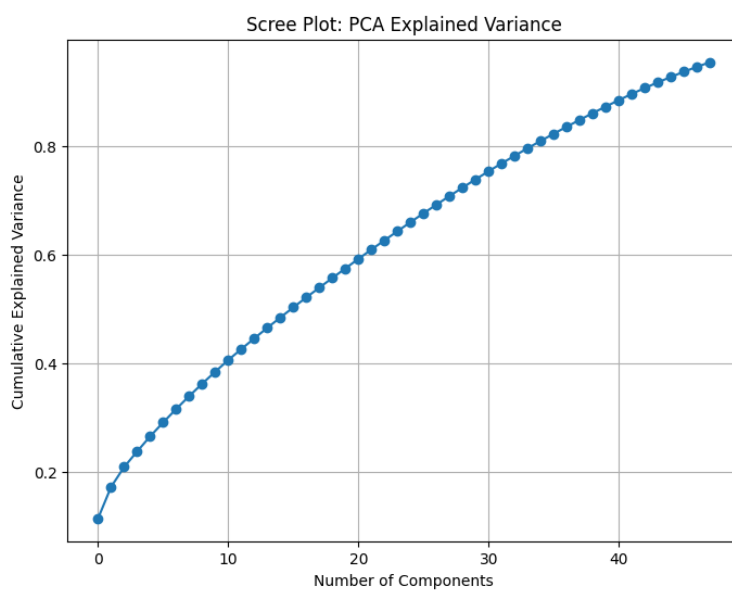


Figure 3: Scree plot of PCA (explained variance ratio)

PCA Summary

Setting	Chosen Components / Variance Target	Explained Variance (%)
With PCA	30 components (95% variance)	95%

Performance Comparison Results

Model	Accuracy (No-PCA)	Accuracy (With-PCA)	Precision (No-PCA)	Precision (With-PCA)
AdaBoost	0.926	0.890	0.938	0.938
Decision Tree	0.926	0.896	0.935	0.935
Gradient Boosting	0.946	0.933	0.962	0.962
KNN	0.891	0.896	0.884	0.884
Logistic Regression	0.920	0.916	0.932	0.932
Naïve Bayes	0.822	0.857	0.723	0.723
Random Forest	0.955	0.936	0.975	0.975
SVM	0.926	0.924	0.935	0.935
Stacking	0.959	0.938	0.976	0.976
XGBoost	0.957	0.941	0.973	0.973

F1 Score Comparison

Model	F1 (No-PCA)	F1 (With-PCA)
AdaBoost	0.910	0.866
Decision Tree	0.911	0.875
Gradient Boosting	0.934	0.918
KNN	0.870	0.875
Logistic Regression	0.902	0.898
Naïve Bayes	0.817	0.842
Random Forest	0.946	0.923
SVM	0.911	0.907
Stacking	0.950	0.926
XGBoost	0.947	0.930

Observations and Analysis

- PCA improved results for **Naïve Bayes** and **KNN**, where dimensionality reduction reduced noise and improved generalization.
- Ensemble models like **Random Forest**, **XGBoost**, and **stacking** performed slightly worse with PCA, since these models already handle high-dimensional data effectively.
- Linear models (**Logistic Regression**, **SVM**) maintained stable performance with and without PCA, showing robustness.
- PCA slightly reduced the variance across folds for models like KNN and Naïve Bayes, leading to more stable results.
- Overall, PCA is beneficial when models are sensitive to high-dimensional noisy features but is not strictly necessary for powerful ensembles.

Conclusion

- PCA is useful for improving simpler models such as Naïve Bayes and KNN by reducing feature noise.
- Ensemble models (Random Forest, XGBoost, Stacking) did not benefit from PCA and sometimes lost accuracy.
- PCA helps to stabilize performance across folds but does not universally improve accuracy.
- The choice of PCA should depend on the dimensionality of the dataset and the sensitivity of the chosen model.